National Autonomous University of Mexico

Faculty of Engineering

Lexical Analyzer

Compilers
Group 05

Team 03

424039512
424153520
321264257
320336658

March 15, 2026

Semester 2026-2

Contents

1 Introduction **2**

2 Theoretical Framework **2**

2.1 Lexical Analysis 2

2.2 Tokenization 2

2.3 Token Classification 3

2.4 Regular Expressions 3

2.5 Finite Automata 4

2.6 Context-Free Grammar 4

2.6.1 CFG proposal 5

3 Development **6**

3.1 List of Keywords 7

3.2 List of regular expressions 7

3.3 System Architecture (Components) 7

3.4 Tokenization Algorithm and Analysis 8

3.5 Metrics and Verification 8

3.6 Integration Tests 9

3.6.1 Test Case 1: Declaration and Basic Operation 9

3.6.2 Test Case 2: Logic with Comments and Strings 9

3.6.3 Test Case 3: Error Handling 10

4 Results **10**

5 Conclusion **16**

6 References **16**

1 Introduction

This project aims to build a lexical analyzer, which represents one of the initial and fundamental stages in the construction of a compiler. This phase is responsible for examining the source code of a program and dividing it into smaller units called tokens, which represent basic elements of a programming language such as keywords, identifiers, operators, constants, and punctuation symbols. The correct functioning of this stage is essential since it prepares the information that will later be used by the subsequent phases of the compiler.
The purpose of this project is to develop a lexical analyzer (lexer) capable of reading a text string or a file containing source code, identifying the different tokens present, and reporting both their classification and the total number found. This process allows the original code to be transformed into a more manageable structure for the later stages of syntactic analysis. In addition to implementing the analyzer, this work also includes the development of a Context-Free Grammar (CFG) that represents one of the examples used as input. To improve the clarity and efficiency of the grammar, the left factoring technique will be applied, which facilitates later processing of the language by a parser.

2 Theoretical Framework

2.1 Lexical Analysis

Lexical analysis is the first phase of a compiler and belongs to the front-end stage. Its purpose is to read the source code written in a high-level language and transform it into a stream of tokens that can be processed by the next compiler phase. In other words, the lexical analyzer scans the input character by character, identifies valid lexemes, and assigns each of them a corresponding token category.

This phase is essential because it provides a structured representation of the source code for syntax analysis. Without lexical analysis, the parser would have to deal directly with raw characters instead of meaningful language units.

2.2 Tokenization

The lexical analyzer reads the sequence of characters that make up a source program and groups them into meaningful units called *lexemes*. For each lexeme, the analyzer generates a token, commonly represented as:

$$\langle \text{token-name}, \text{attribute-value} \rangle$$

This procedure is known as *tokenization*. The token name identifies the category of the

lexeme, while the attribute value may store additional information such as its literal value or an entry in the symbol table. Tokenization is one of the main tasks of the lexical analyzer, since it converts an unstructured text input into a format suitable for further compiler processing.

2.3 Token Classification

The exact classification of tokens depends on the language specification, but in a typical lexical analyzer the main token categories include:

- identifiers,
- operators,
- constants,
- keywords,
- literals,
- punctuators, and
- special characters.

Elements such as whitespace and comments are usually not emitted as tokens, since they do not contribute directly to the syntactic structure of the language. Instead, they are ignored or removed from the output stream during scanning. This classification step allows the compiler to distinguish between the different lexical elements that appear in the source program.

2.4 Regular Expressions

To implement a lexical analyzer, it is necessary to understand how valid strings in a language can be described. Regular expressions provide a formal notation for representing patterns in strings and determining whether a given sequence of characters belongs to a language. A language can be understood as a set of strings, where each string is a finite sequence of symbols taken from a finite alphabet.

The main elements used in regular expressions are the following:

- **Symbol:** any element of the alphabet of the language.
- **Alternation:** indicates a choice between expressions, commonly written with $|$.
- **Concatenation:** represents the sequence of two expressions.

- **Epsilon (ϵ):** represents the empty string.
- **Repetition:** the Kleene star M^* denotes zero or more repetitions of M.

Other common conventions include:

$$M? = (M \mid \epsilon)$$

$$M^+ = M \cdot M^*$$

Regular expressions are therefore useful for describing identifiers, numeric constants, operators, literals, and other token classes. Once these patterns are defined, they can be translated into computational models that recognize them automatically.

2.5 Finite Automata

After defining token patterns through regular expressions, the next step is to represent them in a form that can be executed by a computer. This is achieved through finite automata. A finite automaton consists of a finite set of states, transitions between states labeled with symbols, one start state, and one or more accepting states.

In particular, a *deterministic finite automaton* (DFA) has at most one transition for each symbol from a given state and does not allow epsilon transitions. A DFA processes an input string by starting from the initial state and following exactly one transition for each input character. If, after consuming the full string, the automaton ends in an accepting state, then the string is considered valid; otherwise, it is rejected.

This model is especially important in lexical analysis because it provides a formal and efficient way to recognize tokens. In practice, the lexer implementation in a programming language such as C reproduces the behavior of a DFA through conditional statements, loops, and state transitions encoded in the program logic.

2.6 Context-Free Grammar

Although this project focuses only on lexical analysis, it is important to briefly mention the next compiler phase: syntax analysis. In this phase, the parser uses a context-free grammar (CFG) to determine whether the sequence of tokens produced by the lexical analyzer forms valid statements in the language.

A CFG is composed of:

- an initial nonterminal symbol,
- a set of nonterminal symbols,

- a set of terminal symbols, and
- a set of production rules.

The lexical analyzer does not build the grammar itself, but it supplies the parser with the tokens required for syntactic validation. For this reason, lexical analysis and syntax analysis are closely related: the first identifies the language units, and the second verifies whether those units can be arranged according to the grammar of the language.

2.6.1 CFG proposal

As a proposal for creating the CFG, we decided to use the example of an if-else statement, a basic control structure that contains a good number of programming elements for performing its lexical analysis.

The example code used was:

```
int main() {
    int x = 5;

    if (x > 0) {
        x = x - 1;
    } else {
        x = 0;
    }

    return 0;
}
```

Listing 1: If-Else Conditional Statement

Basic CFG:

$$
\begin{aligned}
\textit{Program} &\rightarrow \texttt{int main ( ) \{}\ \ \textit{Decl IfStmt ReturnStmt} \\
\textit{Decl} &\rightarrow \texttt{int}\ \ \textit{id} = \ \textit{num}\ \texttt{;} \\
\textit{IfStmt} &\rightarrow \texttt{if (}\ \textit{Condition}\ \texttt{)}\ \ \textit{Block} \\
&\quad \mid \texttt{if (}\ \textit{Condition}\ \texttt{)}\ \ \textit{Block}\ \texttt{else}\ \ \textit{Block} \\
\textit{Condition} &\rightarrow \textit{id} > \textit{num} \\
\textit{Block} &\rightarrow \texttt{\{}\ \textit{AssignStmt}\ \texttt{\}} \\
\textit{AssignStmt} &\rightarrow \textit{id} = \textit{Expr}\ \texttt{;} \\
\textit{Expr} &\rightarrow \textit{id} - \textit{num} \\
&\quad \mid \textit{num} \\
\textit{ReturnStmt} &\rightarrow \texttt{return}\ \ \textit{num}\ \texttt{; \}}
\end{aligned}
$$

For the proposed basic CFG, the general syntactic structure of the if-else code example is generated. Taking "Program" as the initial symbol, which represents the entire code fragment to be generated, starting with the main function "int main()", followed by a declaration, a conditional structure, and a return statement. From there, terminals are taken such as "Decl" to declare the variable, "IfStmt" for the control statement, "Condition" for the logical comparison, "Block" for the entire block found inside the if statement, "AssignStmt" for assignments, and "Expr" for arithmetic expressions or numeric values.

CFG with left factoting:

$$
\begin{aligned}
\textit{Program} &\rightarrow \texttt{int main ( ) \{}\ \ \textit{Decl IfStmt ReturnStmt} \\
\textit{Decl} &\rightarrow \texttt{int}\ \ \textit{id} = \ \textit{num}\ \texttt{;} \\
\textit{IfStmt} &\rightarrow \texttt{if (}\ \textit{Condition}\ \texttt{)}\ \ \textit{Block}\ \texttt{else}\ \ \textit{Block} \\
\textit{Condition} &\rightarrow \textit{id} > \textit{num} \\
\textit{Block} &\rightarrow \texttt{\{}\ \textit{AssignStmt}\ \texttt{\}} \\
\textit{AssignStmt} &\rightarrow \textit{id} = \textit{Expr}\ \texttt{;} \\
\textit{Expr} &\rightarrow \textit{id} - \textit{num} \\
&\quad\ \mid \textit{num} \\
\textit{ReturnStmt} &\rightarrow \texttt{return}\ \ \textit{num}\ \texttt{; \}}
\end{aligned}
$$

In the case of the CFG with left factoring, we reorganize the basic grammar mainly in order to eliminate common prefixes in more than one production and thus make the syntactic derivation clearer. For this case, the shared part of the conditional statement "if (Condition) Block", instead of repeating that structure in several productions, is left as a single more compact one. Giving rise to the grammar becoming more organized and thus avoiding redundancy in the definition of the if-else statement.

3 Development

This section describes the technical implementation of the lexical analyzer. The system was developed using the Object-Oriented Programming (OOP) paradigm in Java, prioritizing modularity and the efficient use of regular expressions for pattern recognition. The core of the lexer lies in the formal definition of what constitutes a valid component within the language. The system is designed to decouple the Lexical Grammar (rules) from the Scanning Engine (logic). This is achieved through a multi-class structure:

Decoupling: By encapsulating regex patterns within LexerRule objects, the grammar can be updated or expanded without modifying the core matching algorithm in the Lexer class.

Immutability: The Token class is designed as an immutable data carrier. Once a lexeme is identified and categorized, its state cannot be altered, ensuring data integrity during the subsequent syntax analysis phase.

3.1 List of Keywords

To distinguish between user-defined names and language-specific commands, a Set<String> constant was implemented. The system recognizes the following keywords:

auto	break	case	char
const	continue	default	do
double	else	enum	extern
float	for	goto	if
int	long	register	return
short	signed	sizeof	static
struct	switch	typedef	union
unsigned	void	volatile	while

Table 1: C Language Keywords

3.2 List of regular expressions

The java.util.regex.Pattern class is used to compile the lexemes. The order of these rules in the code is critical for precedence:

- **Punctuators:** `( | ) | { | } | [ | ] | , | ; | . | : | ?`
- **Operators:** `= | == | + | ++ | += | - | - - | -= | -> | * | *= | / | /= | % | %= | & | && | &= | | | || | |= | ! | != | ^ | ~ | < | « | <= | «= | > | » | >= | »= |`
- **Identifiers:** Any name that begins with a letter.
- **Constants:** Any number.
- **Literals:** Any string.

3.3 System Architecture (Components)

The design is divided into five main components that interact to transform the source code into a token list:

- **LexerRule:** The configuration object. It stores the compiled `Pattern`, the associated `TokenType`, and a boolean flag indicating if the token should be discarded.
- **TokenType (Enum):** Provides type safety by defining strict categories: `KEYWORD`, `IDENTIFIER`, `OPERATOR`, `PUNCTUATOR`, `CONSTANT`, `LITERAL`, `EOF`, `ERROR`.

- **Token:** An immutable object acting as the output "package." It contains the token type and the original lexeme.
- **Lexer (Core Logic):** The engine of the system. It contains the `initializeRules()` method to build the pattern hierarchy and the `tokenize()` method to execute the analysis.
- **Main:** Acts as the application controller, handling user input and console output.

3.4 Tokenization Algorithm and Analysis

The processing logic follows a "Region Matching" flow to optimize performance:

- **Pointer Management:** The lexer maintains a `position` variable to track progress. Instead of costly string slicing, it utilizes the `matcher.region(position, inputSize)` method, allowing the engine to analyze only the remaining part of the string.
- **"LookingAt" Principle:** The `lookingAt()` method is employed. Unlike `matches()`, which requires the entire string to fit the pattern, `lookingAt()` only requires a match at the start of the current region.
- **Dynamic Differentiation (Token Promotion):** When a pattern matches an `IDENTIFIER`, the code performs an additional logical check:

  ```
  if(type == TokenType.IDENTIFIER && KEYWORDS.contains(lexeme))
      type = TokenType.KEYWORD;
  ```

  This ensures that words like `int` or `while` are correctly promoted to keywords.
- **Error Recovery:** If no rule matches at the current position, the system captures the problematic character as an `ERROR` type token, reports it, and advances the pointer by one.

3.5 Metrics and Verification

To satisfy the requirement of processing a minimum of **40 tokens**, a static `countTokens(List<Token> tokens)` method was implemented in the `Main` class. This counter performs selective filtering, ignoring `EOF` and `ERROR` tokens to provide an accurate count of valid lexical elements processed.

3.6 Integration Tests

3.6.1 Test Case 1: Declaration and Basic Operation

Input

```
int a = 10;
print(a);
```

Console Output

Position	Token Type	Lexeme
1	KEYWORD	int
2	IDENTIFIER	a
3	OPERATOR	=
4	CONSTANT	10
5	PUNCTUATOR	;
6	IDENTIFIER	print
7	PUNCTUATOR	(
8	IDENTIFIER	a
9	PUNCTUATOR	)
10	PUNCTUATOR	;
Total: 10 Tokens		

3.6.2 Test Case 2: Logic with Comments and Strings

Input

```
// Variable definition
char* msg = "Hello World";
```

Console Output

Position	Token Type	Lexeme
1	KEYWORD	char
2	OPERATOR	*
3	IDENTIFIER	msg
4	OPERATOR	=
5	LITERAL	"Hello World"
6	PUNCTUATOR	;

Note: Comment correctly ignored.

3.6.3 Test Case 3: Error Handling

Input

```
int val = 5 $;
```

Console Output

```
Token{type=KEYWORD, lexeme='int'}
Token{type=IDENTIFIER, lexeme='val'}
Token{type=OPERATOR, lexeme='='}
Token{type=CONSTANT, lexeme='5'}
Token{type=ERROR, lexeme='$'}
Token{type=PUNCTUATOR, lexeme=';'}
```

Analysis Summary:

- **Valid Tokens:** 5
- **Errors Detected:** 1

4 Results

Our lexer can be used in the following manner: by employing a file named test.c, which must contain the code intended to be tested by the lexer.
Below is an example of how a code file (test.c) can be used to test the lexer and obtain a list of the tokens that make up the code. This process is essential for validating the

functionality of the lexer and ensuring that it can correctly identify keywords, operators, and other syntactic elements of the language.

```
int main() {
    int a = 10;
    int b = 20;
    int suma = a + b;
    int resta = a - b;
    int mult = a * b;
    int div = b / a;
    return 0;
}

Tokens Clasification:
#       TYPE            LEXEME
1       KEYWORD         'int'
2       IDENTIFIER      'main'
3       PUNCTUATOR      '('
4       PUNCTUATOR      ')'
5       PUNCTUATOR      '{'
6       KEYWORD         'int'
7       IDENTIFIER      'a'
8       OPERATOR        '='
9       CONSTANT        '10'
10      PUNCTUATOR      ';'
11      KEYWORD         'int'
12      IDENTIFIER      'b'
13      OPERATOR        '='
14      CONSTANT        '20'
15      PUNCTUATOR      ';'
16      KEYWORD         'int'
17      IDENTIFIER      'suma'
18      OPERATOR        '='
19      IDENTIFIER      'a'
20      OPERATOR        '+'
21      IDENTIFIER      'b'
22      PUNCTUATOR      ';'
23      KEYWORD         'int'
24      IDENTIFIER      'resta'
25      OPERATOR        '='
26      IDENTIFIER      'a'
27      OPERATOR        '-'
28      IDENTIFIER      'b'
29      PUNCTUATOR      ';'
30      KEYWORD         'int'
31      IDENTIFIER      'mult'
32      OPERATOR        '='
33      IDENTIFIER      'a'
34      OPERATOR        '*'
35      IDENTIFIER      'b'
36      PUNCTUATOR      ';'
37      KEYWORD         'int'
38      IDENTIFIER      'div'
39      OPERATOR        '='
40      IDENTIFIER      'b'
41      OPERATOR        '/'
42      IDENTIFIER      'a'
43      PUNCTUATOR      ';'
44      KEYWORD         'return'
45      CONSTANT        '0'
46      PUNCTUATOR      ';'
47      PUNCTUATOR      '}'
----------------------------------------------------
Total tokens: 47
```

Figure 1: Tokens in code: Variables and Basic Arithmetic Operations

```
Input:
int main() {
    int x = 5;

    if (x > 0) {
        x = x - 1;
    } else {
        x = 0;
    }
    return 0;
}

Tokens Clasification:
#       TYPE            LEXEME
1       KEYWORD         'int'
2       IDENTIFIER      'main'
3       PUNCTUATOR      '('
4       PUNCTUATOR      ')'
5       PUNCTUATOR      '{'
6       KEYWORD         'int'
7       IDENTIFIER      'x'
8       OPERATOR        '='
9       CONSTANT        '5'
10      PUNCTUATOR      ';'
11      KEYWORD         'if'
12      PUNCTUATOR      '('
13      IDENTIFIER      'x'
14      OPERATOR        '>'
15      CONSTANT        '0'
16      PUNCTUATOR      ')'
17      PUNCTUATOR      '{'
18      IDENTIFIER      'x'
19      OPERATOR        '='
20      IDENTIFIER      'x'
21      OPERATOR        '-'
22      CONSTANT        '1'
23      PUNCTUATOR      ';'
24      PUNCTUATOR      '}'
25      KEYWORD         'else'
26      PUNCTUATOR      '{'
27      IDENTIFIER      'x'
28      OPERATOR        '='
29      CONSTANT        '0'
30      PUNCTUATOR      ';'
31      PUNCTUATOR      '}'
32      KEYWORD         'return'
33      CONSTANT        '0'
34      PUNCTUATOR      ';'
35      PUNCTUATOR      '}'
----------------------------------------------------
Total tokens: 35
```

Figure 2: Tokens in code: If-Else Conditional Statement

```
Input:
int main() {
    int i = 0;
    while (i < 10) {
        i++;
    }
    return 0;
}

Tokens Clasification:
#       TYPE            LEXEME
1       KEYWORD         'int'
2       IDENTIFIER      'main'
3       PUNCTUATOR      '('
4       PUNCTUATOR      ')'
5       PUNCTUATOR      '{'
6       KEYWORD         'int'
7       IDENTIFIER      'i'
8       OPERATOR        '='
9       CONSTANT        '0'
10      PUNCTUATOR      ';'
11      KEYWORD         'while'
12      PUNCTUATOR      '('
13      IDENTIFIER      'i'
14      OPERATOR        '<'
15      CONSTANT        '10'
16      PUNCTUATOR      ')'
17      PUNCTUATOR      '{'
18      IDENTIFIER      'i'
19      OPERATOR        '++'
20      PUNCTUATOR      ';'
21      PUNCTUATOR      '}'
22      KEYWORD         'return'
23      CONSTANT        '0'
24      PUNCTUATOR      ';'
25      PUNCTUATOR      '}'
-----------------------------------------------------
Total tokens: 25
```

Figure 3: Tokens in code: While Loop

```
Input:
int main() {
    int i;

    for (i = 0; i < 5; i++) {
    }

    return 0;
}

Tokens Clasification:
#       TYPE            LEXEME
1       KEYWORD         'int'
2       IDENTIFIER      'main'
3       PUNCTUATOR      '('
4       PUNCTUATOR      ')'
5       PUNCTUATOR      '{'
6       KEYWORD         'int'
7       IDENTIFIER      'i'
8       PUNCTUATOR      ';'
9       KEYWORD         'for'
10      PUNCTUATOR      '('
11      IDENTIFIER      'i'
12      OPERATOR        '='
13      CONSTANT        '0'
14      PUNCTUATOR      ';'
15      IDENTIFIER      'i'
16      OPERATOR        '<'
17      CONSTANT        '5'
18      PUNCTUATOR      ';'
19      IDENTIFIER      'i'
20      OPERATOR        '++'
21      PUNCTUATOR      ')'
22      PUNCTUATOR      '{'
23      PUNCTUATOR      '}'
24      KEYWORD         'return'
25      CONSTANT        '0'
26      PUNCTUATOR      ';'
27      PUNCTUATOR      '}'
--------------------------------------------------
Total tokens: 27
```

Figure 4: Tokens in code: For Loop

```
Input:
int main() {
    int n = 10;
    int a = 0, b = 1, c;
    int i;
    for (i = 0; i < n; i++) {
        printf("%d ", a);
        c = a + b;
        a = b;
        b = c;
    }
    return 0;
}

Tokens Clasification:
#       TYPE            LEXEME
1       KEYWORD         'int'
2       IDENTIFIER      'main'
3       PUNCTUATOR      '('
4       PUNCTUATOR      ')'
5       PUNCTUATOR      '{'
6       KEYWORD         'int'
7       IDENTIFIER      'n'
8       OPERATOR        '='
9       CONSTANT        '10'
10      PUNCTUATOR      ';'
11      KEYWORD         'int'
12      IDENTIFIER      'a'
13      OPERATOR        '='
14      CONSTANT        '0'
15      PUNCTUATOR      ','
16      IDENTIFIER      'b'
17      OPERATOR        '='
18      CONSTANT        '1'
19      PUNCTUATOR      ','
20      IDENTIFIER      'c'
21      PUNCTUATOR      ';'
22      KEYWORD         'int'
23      IDENTIFIER      'i'
24      PUNCTUATOR      ';'
25      KEYWORD         'for'
26      PUNCTUATOR      '('
27      IDENTIFIER      'i'
28      OPERATOR        '='
29      CONSTANT        '0'
30      PUNCTUATOR      ';'
31      IDENTIFIER      'i'
32      OPERATOR        '<'
33      IDENTIFIER      'n'
34      PUNCTUATOR      ';'
35      IDENTIFIER      'i'
36      OPERATOR        '++'
37      PUNCTUATOR      ')'
38      PUNCTUATOR      '{'
39      IDENTIFIER      'printf'
40      PUNCTUATOR      '('
41      LITERAL         '"%d "'
42      PUNCTUATOR      ','
43      IDENTIFIER      'a'
44      PUNCTUATOR      ')'
45      PUNCTUATOR      ';'
46      IDENTIFIER      'c'
47      OPERATOR        '='
48      IDENTIFIER      'a'
49      OPERATOR        '+'
50      IDENTIFIER      'b'
51      PUNCTUATOR      ';'
52      IDENTIFIER      'a'
53      OPERATOR        '='
54      IDENTIFIER      'b'
55      PUNCTUATOR      ';'
56      IDENTIFIER      'b'
57      OPERATOR        '='
58      IDENTIFIER      'c'
59      PUNCTUATOR      ';'
60      PUNCTUATOR      '}'
61      KEYWORD         'return'
62      CONSTANT        '0'
63      PUNCTUATOR      ';'
64      PUNCTUATOR      '}'
-----------------------------------------------
Total tokens: 64
```

Figure 5: Tokens in code: Iterative Fibonacci

5 Conclusion

The results of our project show the classification of each token and the total count found within the source code. To reach this point it was important for our team to understand how to implement the process of tokenization in Java. The regular expressions we utilized gave us the formal notion of how to analyze each string to classify it into categories like keywords, identifiers or operators. Moreover we built a logic that reproduces the behavior of a deterministic finite automaton (DFA) to provide a formalism that helped us code the language rules.

Then we used Java to recreate this DFA behavior mostly using modular classes and conditional logic to manage state transitions and pointers to track our position in the input string. By applying the "LookingAt" principle and Region Matching, we ensured the engine could efficiently identify tokens at the start of a region and promote identifiers to keywords when necessary. Finally we verified that our tokens can be represented by a Context-Free Grammar (CFG) which is vital because the next phase of the compiler syntax analysis requires a CFG to work. As we can see the way we built this lexical analyzer was based on our theoretical framework successfullly using these concepts to implement a functional tokenization process.

6 References

[1] GeeksforGeeks, Working of lexical analyzer in compiler, https://www.geeksforgeeks.org/compiler-design/working-of-lexical-analyzer-in-compiler/, Accessed: 2026-03-13.

[2] Oracle, Regular Expressions, https://docs.oracle.com/en/cloud/paas/data-safe/udscs/regular-expressions.html, Accessed: 2026-03-14.

[3] Oracle, Pattern-matching Conditions, https://docs.oracle.com/en/database/oracle/oracle-database/19/sqlrf/Pattern-matching-Conditions.html, Accessed: 2026-03-14.

[4] Oracle, Matcher (Java Platform SE 8), https://docs.oracle.com/javase/8/docs/api/java/util/regex/Matc
Accessed: 2026-03-14.

[5] GeeksforGeeks, Token, Patterns and Lexemes, https://www.geeksforgeeks.org/compiler-design/token-patterns-and-lexems/, Accessed: 2026-03-13.

[6] GeeksforGeeks, Introduction of Lexical Analysis, https://www.geeksforgeeks.org/compiler-design/introduction-of-lexical-analysis/, Accessed: 2026-03-13.

[7] W3Schools, Java RegEx, https://www.w3schools.com/java/java$_{r}egex.asp, Accessed : 2026 - 03 - 15.$

[8] Neso Academy, Compiler Design (Playlist), https://www.youtube.com/playlist?list=PLBlnK6fEyqRjT3
LFY-q, Accessed: 2026-03-12.